

# 网页文章错误检测 Agent —— 需求分析文档

**项目：**网页文章错误检测 Agent（浏览器插件）  
**课程：**软件工程与计算III · 迭代三  
**文档定位：**本文档是迭代三的需求分析文档。迭代二已有基础架构、SSE 接口、4 个基础工具等  
本文不再重复。

## 1. 概述

### 1.1 迭代三目标

迭代二结束时系统具备单 Agent 核查流水线（扫描 → ReAct → 输出标注），迭代三围绕以下 7 个功能目标展开：

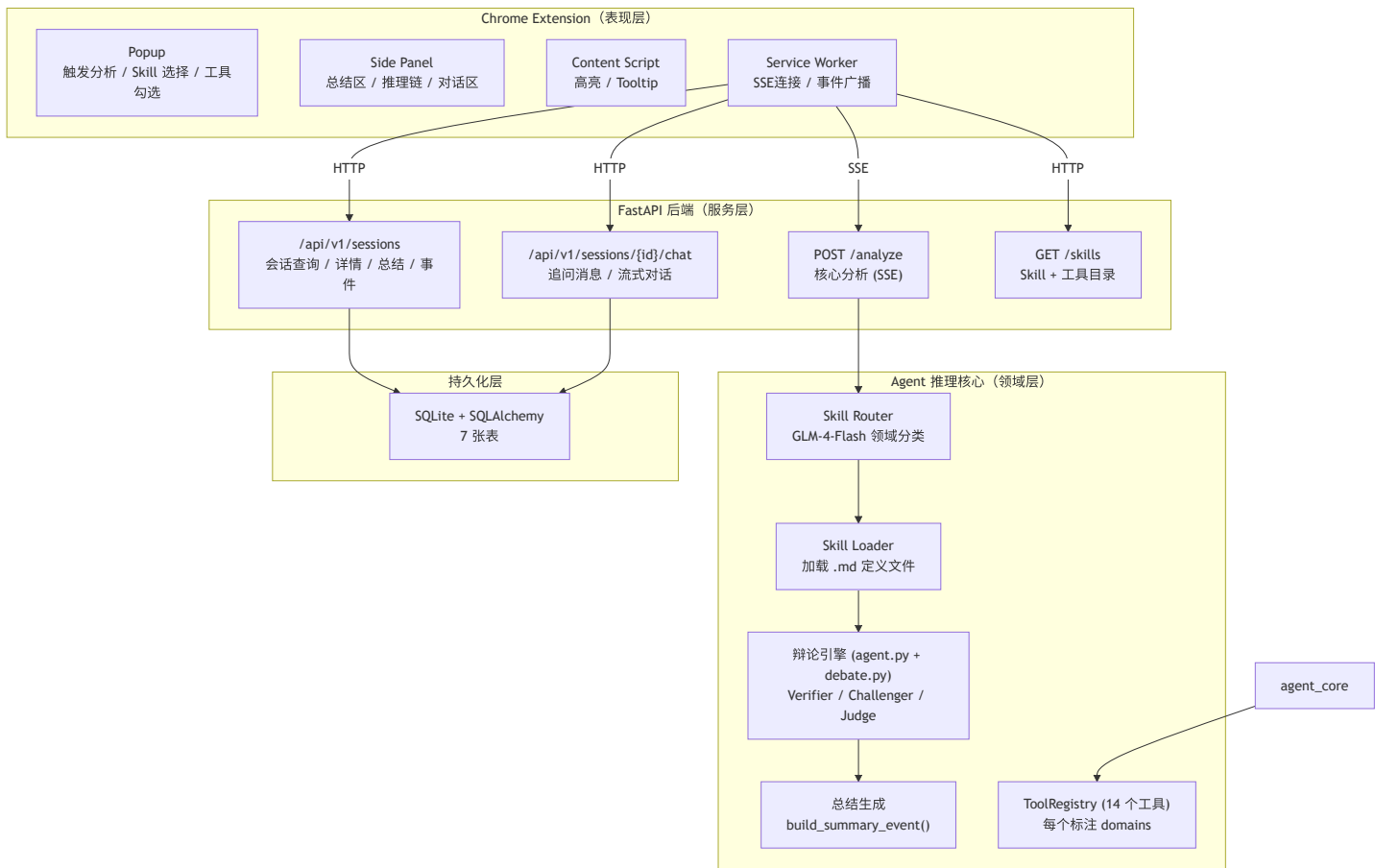
| 编号 | 功能目标       | 主要模块                                                |
|----|------------|-----------------------------------------------------|
| F1 | 多 Agent 辩论 | agent/agent.py 、 agent/debate.py                    |
| F2 | 分析总结       | agent/debate.py:build_summary_event()               |
| F3 | 领域识别与路由    | agent/skills/router.py 、 agent/skills/defs/*.md     |
| F4 | 流式对话与追问    | agent/agent.py:chat() 、 backend/main.py:chat_stream |
| F5 | 领域工具扩展     | agent/tools/ 新增 10 个工具                              |
| F6 | Skill 系统   | agent/skills/ (Markdown 文件 + YAML frontmatter)      |
| F7 | 数据库落库      | backend/db/ (7 张表)                                  |

# 1.2 文档结构

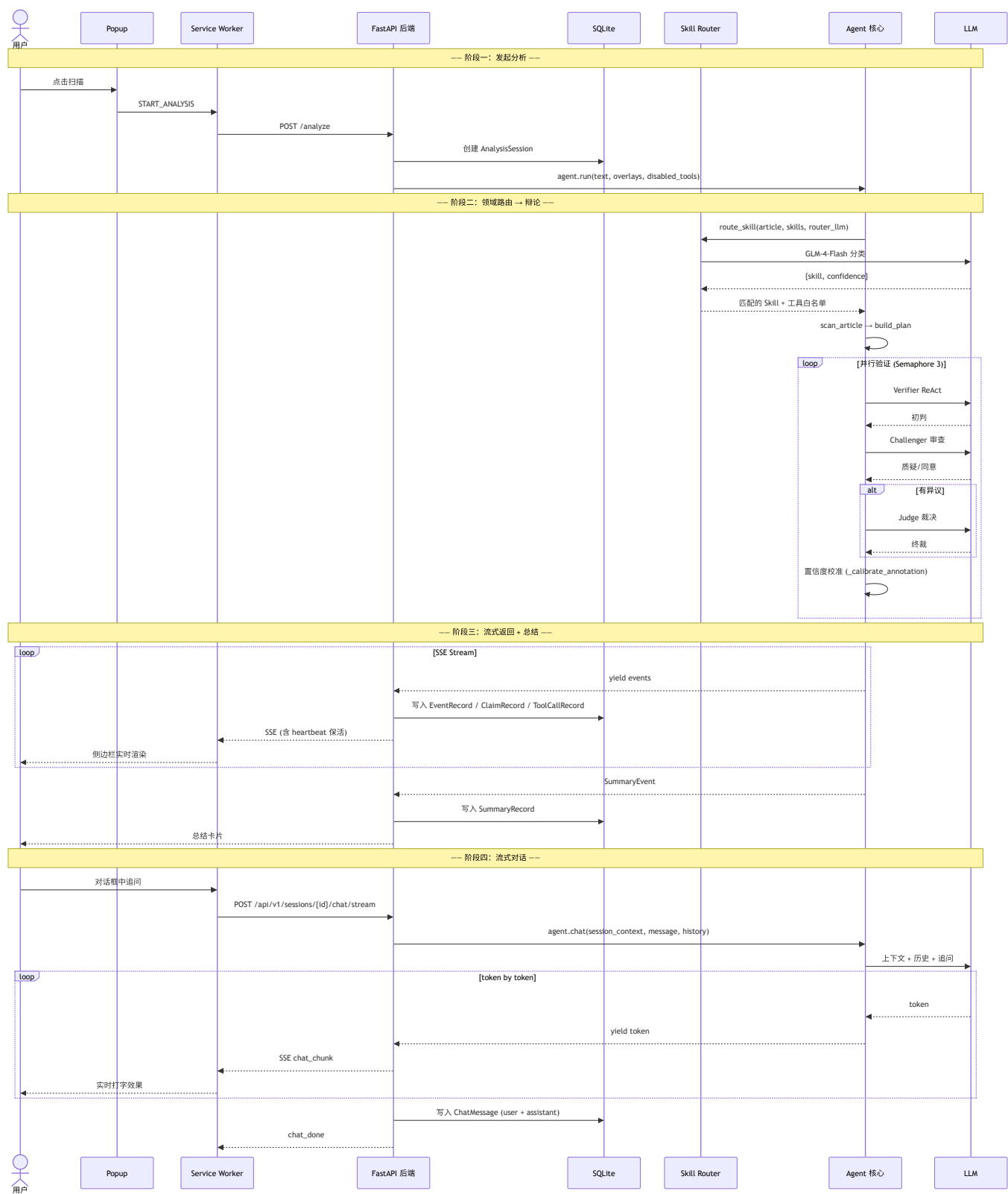
- 1. 概述
- 2. 系统架构演进
- 3. 用例分析
- 4. Agent 辩论系统
- 5. 分析总结
- 6. 领域识别与 Skill 路由
- 7. 流式对话与追问
- 8. 领域工具扩展
- 9. Skill 管理系统
- 10. 数据库设计

## 2. 系统架构演进

### 2.1 产品化后的分层架构

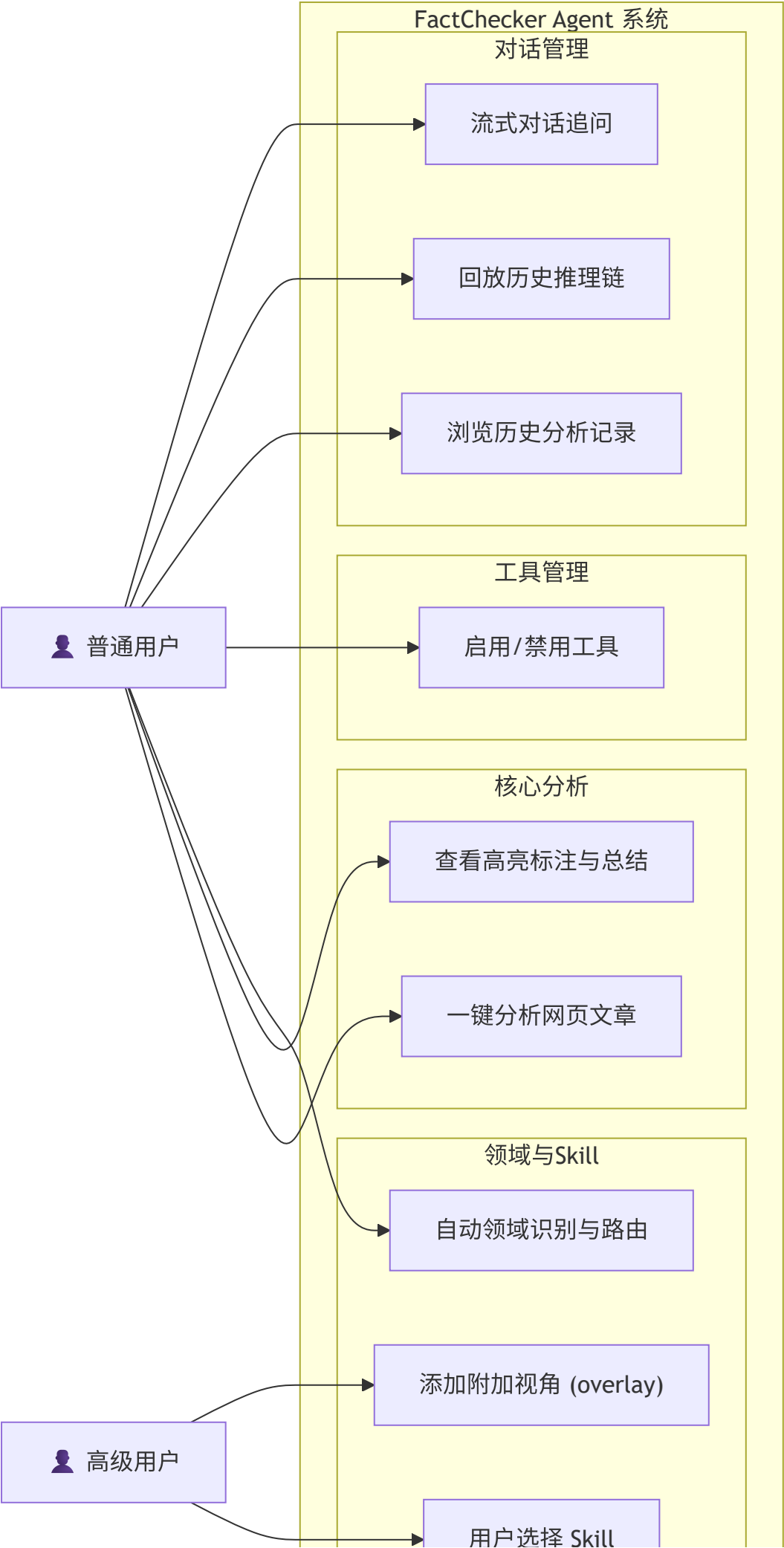


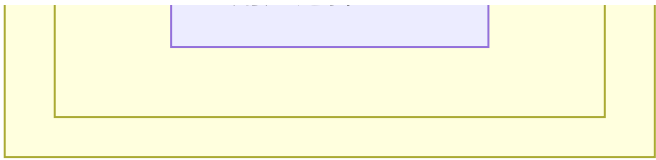
2.2 数据流全景



# 3. 用例分析

## 3.1 用例图





### 3.2 核心用例详述

#### UC-01：一键分析网页文章（自动领域路由）

| 项目   | 内容                                                                                                           |
|------|--------------------------------------------------------------------------------------------------------------|
| 参与者  | 普通用户                                                                                                         |
| 前置条件 | 用户在浏览器中打开一篇包含正文的网页                                                                                           |
| 主流程  | 1. 点击扩展图标 → 扫描 2. 后端自动进行领域识别（GLM-4-Flash 分类 → 选取 Skill） 3. Agent 并行验证声明（辩论模式） 4. 侧边栏实时展示辩论推理链 5. 分析结束后推送总结卡片 |
| 后置条件 | 页面错误声明高亮，侧边栏展示完整辩论过程和总结                                                                                      |

#### UC-02：查看分析总结

| 项目   | 内容                                                                                                 |
|------|----------------------------------------------------------------------------------------------------|
| 参与者  | 普通用户                                                                                               |
| 前置条件 | 完成一次文章分析                                                                                           |
| 主流程  | 1. 分析结束侧边栏展示总结卡片 2. 卡片包含：错误条数/通过条数、error_breakdown、challenged/revised 统计、代表性声明、整体结论 3. 历史会话可重新查看总结 |
| 后置条件 | 用户可获得本次分析的结构化摘要                                                                                    |

#### UC-03：在对话框中流式追问

| 项目   | 内容                                                                                  |
|------|-------------------------------------------------------------------------------------|
| 参与者  | 普通用户                                                                                |
| 前置条件 | 完成一次文章分析                                                                            |
| 主流程  | 1. 在侧边栏对话框输入问题 2. Agent 基于原文章、声明验证结果、总结回答 3. 回复逐 token 流式展示（打字效果） 4. 对话历史持久化，刷新后可恢复 |
| 后置条件 | 用户可通过对话深入理解分析结果                                                                     |

# 4. Agent 辩论系统

实现文件：[agent/agent/agent.py](#)（主控循环）、[agent/agent/debate.py](#)（辩论辅助）

## 4.1 辩论模型

| 角色               | 职责                                             | 特点              |
|------------------|------------------------------------------------|-----------------|
| Verifier Agent   | 调用工具获取证据，给出初判（error_type、confidence、reasoning） | 必须调用工具，必须明确判断   |
| Challenger Agent | 审查 Verifier 的证据链和推理，决定支持或质疑                    | 不能调工具，只基于已有信息判断 |
| Judge Agent      | 审阅双方论据做出最终裁决                                   | 输出可直接持久化的结论     |

**高置信快速通道：**Verifier 置信度  $\geq 0.9$  且无工具/格式错误时，跳过 Challenger 和 Judge 直接采纳，节省 LLM 调用。

**置信度校准**（\_calibrate\_annotation）：Judge 终裁后根据客观信号（工具调用次数、错误、证据 URL 数量）调整置信度。低于 0.60 的发现将被忽略（error\_type  $\rightarrow$  None）。

**Challenger 异议重验证：**Challenger 提出异议（stance="challenge"）后触发第 2 轮 Verifier 重验证，Challenger 给出 missing\_evidence 和 suggested\_queries 引导方向。

## 4.2 辩论生命周期

```
Coordinator 发起首轮辩论 (DebateEvent, phase=started, role=coordinator)
|
├─ Verifier 初判 (ReAct 循环)
|   └─ DebateEvent(phase=argument, role=verifier)
|
├─ confidence >= 0.9 且无错误?
|   YES → 快速通道: DebateEvent(phase=result) → 校准 → 输出
|   NO  ↓
|
├─ Challenger 审查
|   └─ DebateEvent(phase=argument, role=challenger, stance=support|challenge)
|
├─ Challenger 反对?
|   YES → 第 2 轮 ReAct (rebuttal) → DebateEvent(phase=argument, role=verifier)
|
└─ Judge 裁决
    └─ DebateEvent(phase=result, role=judge) → 置信度校准 → 输出 AnnotationEvent
```

## 4.3 辩论事件模型

辩论使用统一的 DebateEvent ([agent/agent/models.py](#)):

| 字段            | 类型                                                     | 说明                                     |
|---------------|--------------------------------------------------------|----------------------------------------|
| type          | "debate"                                               | 固定值                                    |
| claim_id      | str                                                    | 关联声明                                   |
| round         | int                                                    | 辩论轮次 (1 或 2)                           |
| phase         | "started" / "argument" / "result"                      | 辩论阶段                                   |
| role          | "coordinator" / "verifier" /<br>"challenger" / "judge" | 发言角色                                   |
| stance        | "support" / "challenge" / "neutral"                    | 立场                                     |
| confidence    | float                                                  | 置信度                                    |
| evidence_urls | list[str]                                              | 证据链接                                   |
| details       | dict                                                   | 额外信息 (fast_path、<br>reverify_target 等) |

## 4.4 需求清单

| 需求编号    | 需求描述                                                          |
|---------|---------------------------------------------------------------|
| DEB-001 | Verifier ReAct → Challenger 审查 → Judge 裁决三阶段辩论                |
| DEB-002 | 三套角色 System Prompt (Verifier 证据导向、Challenger 质疑导向、Judge 裁决导向) |
| DEB-003 | 统一 DebateEvent , 三阶段 phase=started/argument/result            |
| DEB-004 | 高置信快速通道 (confidence ≥ 0.9 直接采纳)                               |
| DEB-005 | Challenger 异议时触发第 2 轮重验证 (rebuttal)                           |
| DEB-006 | 置信度校准 (_calibrate_annotation), 低于 0.60 忽略                     |

| 需求编号    | 需求描述                                                                |
|---------|---------------------------------------------------------------------|
| DEB-007 | 并行验证（ <code>asyncio.Semaphore 3</code> ），通过 <code>Queue</code> 汇聚事件 |

## 5. 分析总结

实现文件：[agent/agent/debate.py](#) `build_summary_event()`

### 5.1 SummaryEvent 数据模型

```
class SummaryEvent(BaseModel):
    type: Literal["summary"] = "summary"
    total_claims: int           # 总声明数
    total_annotations: int      # 发现错误数
    clean_claims: int           # 通过数
    challenged_claims: int      # 被 Challenger 质疑的声明数
    revised_claims: int         # 初判被修正的声明数
    error_breakdown: dict[str, int] # {"factual_error": 2, "logical_fallacy": 1, ...}
    representative_claims: list[dict] # 代表性声明（最高置信度前 3 条）
    overall_conclusion: str       # 整体结论
    reverify_supported: bool     # 是否支持重验证
```

### 5.2 需求清单

| 需求编号    | 需求描述                                                                      |
|---------|---------------------------------------------------------------------------|
| SUM-001 | done 事件前推送 summary 事件，包含统计 + 代表性声明 + 整体结论                                 |
| SUM-002 | build_summary_event() 汇总全部 ClaimDebateRecord 生成结构化总结                      |
| SUM-003 | 侧边栏展示总结卡片（可折叠），展示各维度统计                                                    |
| SUM-004 | done 事件扩展 total_claims、summary_available、reverify_supported、claim_results |

## 6. 领域识别与 Skill 路由

实现文件：[agent/skills/router.py](#)、[agent/agent/skills/base.py](#)

### 6.1 路由机制

- 使用 GLM-4-Flash 对文章前 800 字符做领域分类
- 在 `agent/skills/defs/*.md` 中定义的 domain skill 中选择最匹配的
- 置信度 < 0.5 时回退到 `general`
- 用户可通过 `disabled_tools` 参数对路由结果做减法（只减不增）

Skill 类型：

- `domain`：领域档案，互斥，路由只选 1 个（内置全是 domain）
- `overlay`：附加视角，由用户开关，可多个叠加

### 6.2 领域标签与路由

| 标签          | 说明              | 示例工具                                                                             |
|-------------|-----------------|----------------------------------------------------------------------------------|
| medical     | 医学、药物、疾病        | pubmed_scientific_search、consumer_health_verifier                                |
| finance     | 经济、金融、市场        | macro_statistics_global、stock_market_quotes                                      |
| technology  | 科技、IT、科研        | academic_paper_search、preprint_arxiv_search、patent_status_lookup、wikidata_lookup |
| news_policy | 新闻、政策           | fact_check_domestic、fact_check_global                                            |
| general     | 兜底（路由失败 / 单领域时） | web_search、wikipedia_lookup、source_verifier、cross_reference                      |

**领域工具白名单：**每个 Skill 的 `allowed_tools` 定义可用工具子集。Agent 的 `_react_loop` 在工具调用前检查 `tool_name in skill.allowed_tools`，不在白名单的工具拒绝调用并提示 Agent 换工具。

### 6.3 需求清单

| 需求编号    | 需求描述                                                    |
|---------|---------------------------------------------------------|
| DOM-001 | GLM-4-Flash 对文章前 800 字符做领域分类，低置信回退 <code>general</code> |
| DOM-002 | 路由后根据 <code>Skill.allowed_tools</code> 启用领域工具白名单        |
| DOM-003 | Skill 领域 System Prompt 注入 ReAct                         |
| DOM-004 | <code>disabled_tools</code> 参数支持用户对路由结果做减法              |

| 需求编号    | 需求描述                          |
|---------|-------------------------------|
| DOM-005 | overlay 支持：用户可叠加多个附加视角 prompt |

## 7. 流式对话与追问

实现文件：[agent/agent/agent.py](#) chat()、[backend/main.py](#) chat\_stream

### 7.1 对话系统架构

后端 SSE 流式接口：POST /api/v1/sessions/{session\_id}/chat/stream

- 请求：{"message": "...", "related\_claim\_id": null, "mode": null}
- 响应：SSE 流，逐 token 推送 chat\_chunk 事件，结束推送 chat\_done
- 处理：build\_chat\_context(db, session\_id) 构建包含原文章、声明验证结果、总结的上下文，注入对话历史（最近 20 条），agent.chat() 流式生成回复

REST 接口 ([backend/routers/sessions.py](#)):

- POST /api/v1/sessions/{id}/chat — 发送追问消息
- GET /api/v1/sessions/{id}/chat — 拉取对话历史
- POST /api/v1/sessions/{id}/recheck — 提交重验证请求

对话上下文构建 ([backend/services/context\\_builder.py](#)): 从 AnalysisSession + ClaimRecord + SummaryRecord 聚合上下文，含文章标题、正文（截断 3000 字）、声明验证结果（含判定/错误类型/置信度/推理/证据）、分析总结。

### 7.2 需求清单

| 需求编号     | 需求描述                                              |
|----------|---------------------------------------------------|
| CHAT-001 | SSE 流式对话接口 POST /api/v1/sessions/{id}/chat/stream |
| CHAT-002 | 对话上下文构建（文章 + 声明 + 总结 + 历史）                        |
| CHAT-003 | agent.chat() 流式生成，ChatChunkEvent + ChatDoneEvent  |
| CHAT-004 | ChatMessage 持久化（user + assistant）                 |
| CHAT-005 | REST 接口：发送消息、拉取历史、提交重验证                           |
| CHAT-006 | 侧边栏对话框 UI                                         |

## 8. 领域工具扩展

实现文件: [agent/tools/registry.py](#)

### 8.1 工具总览 (14 个)

| 工具名                      | 领域                   | 功能                         |
|--------------------------|----------------------|----------------------------|
| web_search               | general              | Tavily/DuckDuckGo 网络搜索     |
| wikipedia_lookup         | general              | 中英文维基百科查询                  |
| source_verifier          | general              | 来源可信度验证 (readability 提取正文) |
| cross_reference          | general              | 文章内部语义检索 + 矛盾检测            |
| wikidata_lookup          | general, technology  | Wikidata 结构化实体属性查询         |
| macro_statistics_global  | finance              | 全球宏观经济与社会统计数据核查            |
| stock_market_quotes      | finance              | 全球个股实时行情查询 (A股/美股/港股)      |
| pubmed_scientific_search | medical              | PubMed 专业医学文献检索            |
| consumer_health_verifier | medical              | 日常健康常识与非处方药核查              |
| academic_paper_search    | technology           | Semantic Scholar 同行评议文献查询  |
| preprint_arxiv_search    | technology           | arXiv 预印本论文检索              |
| patent_status_lookup     | technology           | 全球专利法律状态与授权状态查询            |
| fact_check_domestic      | news_policy, general | 国内辟谣平台记录检索                 |
| fact_check_global        | news_policy, general | 国际事实核查平台记录检索               |

每个 ToolSpec 新增 domains 字段标记适用领域。

### 8.2 需求清单

| 需求编号         | 需求描述                                     |
|--------------|------------------------------------------|
| TOOL-NEW-001 | wikidata_lookup (Wikidata API)           |
| TOOL-NEW-002 | macro_statistics_global (World Bank API) |
| TOOL-NEW-003 | stock_market_quotes (实时行情)               |
| TOOL-NEW-004 | pubmed_scientific_search (NCBI Entrez)   |

| 需求编号         | 需求描述                                    |
|--------------|-----------------------------------------|
| TOOL-NEW-005 | consumer_health_verifier（日常健康）          |
| TOOL-NEW-006 | academic_paper_search（Semantic Scholar） |
| TOOL-NEW-007 | preprint_arxiv_search（arXiv API）        |
| TOOL-NEW-008 | patent_status_lookup（专利查询）              |
| TOOL-NEW-009 | fact_check_domestic（国内辟谣）               |
| TOOL-NEW-010 | fact_check_global（国际事实核查）               |

## 9. Skill 管理系统

实现文件：[agent/skills/base.py](#)、[agent/skills/defs/\\*.md](#)

### 9.1 Skill 数据模型

Skill 以 **Markdown 文件** 形式存放在 `agent/skills/defs/` 目录，由 `YAML frontmatter` + 正文构成：

```
---
name: medical
description: 医学健康类文章的核查—涉及药物、疾病、临床试验等声明
allowed_tools: [pubmed_scientific_search, consumer_health_verifier, web_search, cross_reference]
kind: domain
---

你是专业的医学事实核查 Agent。
优先查阅 PubMed、WHO 等权威医学来源。
对药物剂量声明严格数值比对，误差不超过 5%。
...（正文作为 System Prompt 注入 ReAct）
```

**Skill 类型**（`kind` 字段）：

- `domain`：领域档案，互斥（路由只选 1 个）
- `overlay`：附加视角，由用户开关，多个可叠加

**overlay 安全限制**：`name` ≤ 40 字符，`prompt` ≤ 4000 字符

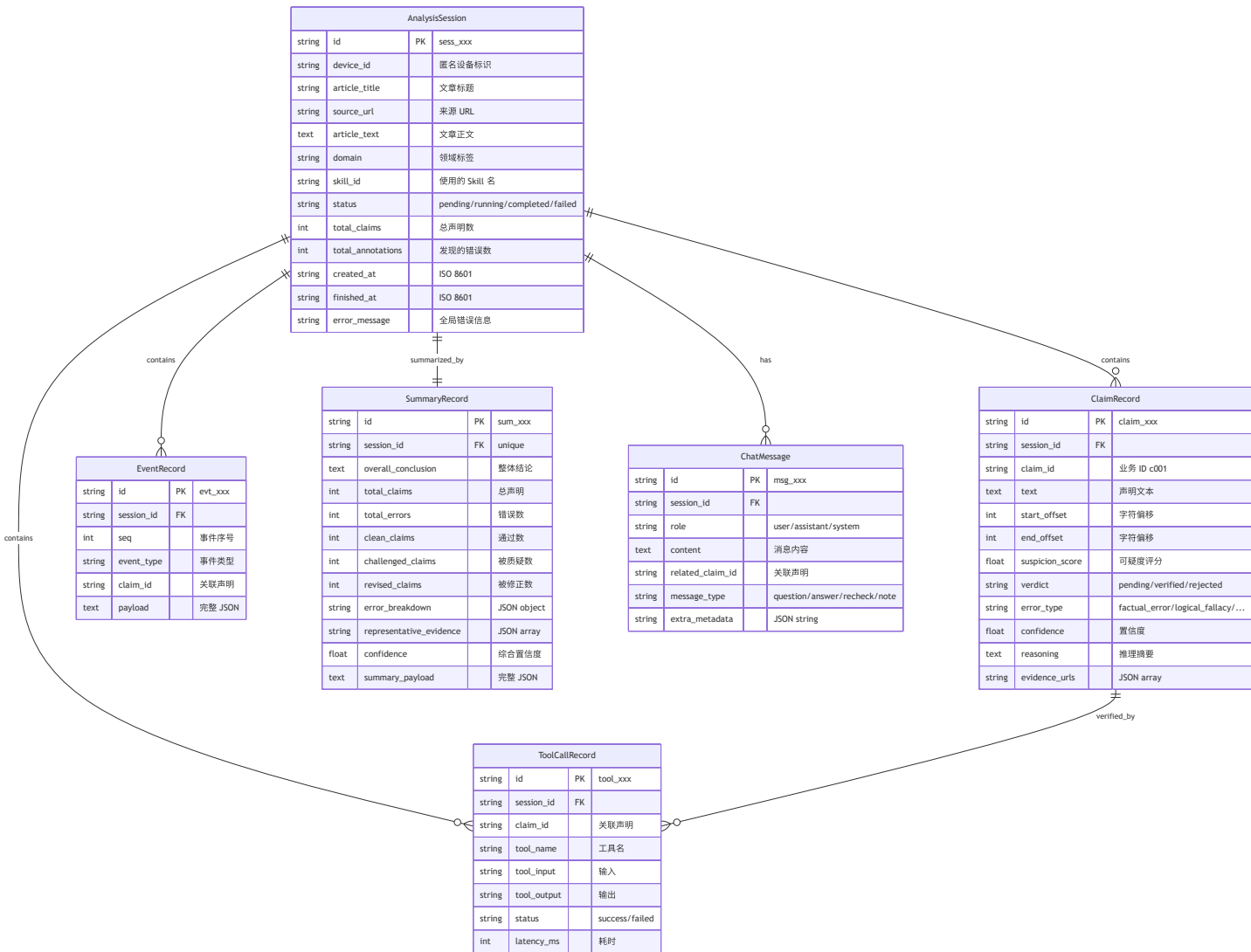
## 9.2 需求清单

| 需求编号      | 需求描述                                                               |
|-----------|--------------------------------------------------------------------|
| SKILL-001 | Markdown + YAML frontmatter 格式定义 Skill                             |
| SKILL-002 | load_skills() 自动扫描 defs/ 目录加载所有 *.md                               |
| SKILL-003 | 强制要求 general 兜底 Skill， allowed_tools 引用校验                          |
| SKILL-004 | route_skill() GLM-4-Flash 领域路由                                     |
| SKILL-005 | build_overlay_skill() 从用户传入数据构造 overlay Skill                      |
| SKILL-006 | 内置 Skill 库（general / medical / finance / technology / news_policy） |
| SKILL-007 | GET /skills 返回领域 + 工具目录（含 used_by 反查）                              |

## 10. 数据库设计

实现文件：[backend/db/models.py](#)

10.1 7 张表



10.2 落库策略

- /analyze 开始时创建 AnalysisSession (status=pending)
- SSE 流中每个事件通过 \_persist\_agent\_event() 写入 EventRecord + 更新对应表
- plan 事件 → 创建 ClaimRecord + 更新 session 状态为 running
- annotation 事件 → 更新对应 ClaimRecord (error\_type、confidence、reasoning、evidence\_urls)
- tool\_call 事件 → 写入 ToolCallRecord
- debate 事件 → 写入 EventRecord , phase=result 时更新 ClaimRecord
- summary 事件 → 写入 SummaryRecord
- done 事件 → 更新 session 状态为 completed, 兜底构建 SummaryRecord
- error 事件 (无 claim\_id) → 更新 session 状态为 failed

10.3 心跳保活机制

SSE 流单次 await agent.run().\_\_anext\_\_() 可能超过 Chrome MV3 Service Worker 的 30s 空闲超时。后端通过 15s 定时器检测 Agent 输出：若 15s 内无事件，自动推送

`{"type": "status", "stage": "heartbeat", "message": "keepalive"}` 保持连接。

## 10.4 需求清单

| 需求编号   | 需求描述                                                                                                      |
|--------|-----------------------------------------------------------------------------------------------------------|
| DB-001 | SQLite + SQLAlchemy ORM，启动时自动建表                                                                           |
| DB-002 | 7 张表 ORM 模型（AnalysisSession、ClaimRecord、EventRecord、ToolCallRecord、SummaryRecord、ChatMessage、SkillRecord） |
| DB-003 | _persist_agent_event() 全链路落库                                                                              |
| DB-004 | REST API：session CRUD / events / claims / summary / chat                                                  |
| DB-005 | SSE 心跳保活机制（15s heartbeat）                                                                                 |